

WorkBridge AI

Guardrailed Multi-Agent Enterprise Workflow Assistant

Junior FDE Pre-screening Assignment | Wipro | May 2026

1. Project Overview

Enterprise support teams frequently field the same policy-related questions repeatedly — questions about refund approval thresholds, how to handle personally identifiable information, or when to escalate a compliance issue. Relying on keyword searches over static documents is slow and error-prone, and general-purpose AI assistants risk generating answers from training data rather than the organisation's actual policies.

WorkBridge AI addresses this by providing a structured, guardrailed AI assistant that answers employee policy questions using only retrieved internal policy documents. The goal was to demonstrate that a practical multi-agent architecture can reduce response time, improve consistency, and maintain answer reliability without requiring complex infrastructure. The project was built as a working prototype within a short development timeline, using a lightweight but defensible design.

2. Solution Overview

WorkBridge AI is a Streamlit-based web application that routes each user query through a controlled sequential pipeline of six specialised agents. Each agent has a clearly defined responsibility, and no agent step is skipped. The system uses OpenAI's GPT-4o-mini API for language generation tasks while keeping retrieval and guardrail logic entirely within Python, without any dependency on external vector databases or orchestration frameworks.

Every agent's output is displayed in the UI, giving users full visibility into how the system arrived at its answer. This traceability was a deliberate design choice: in an enterprise context, explainability matters as much as correctness.

3. Architecture & Agent Workflow

The pipeline executes agents in a fixed sequential order. A query that is blocked at the Guardrail stage never reaches the language model, keeping the system both safe and cost-efficient.

Guardrail Agent

Runs before any LLM call. Uses regex-based pattern matching to detect prompt injection attempts (e.g. "ignore previous instructions", "bypass security"), jailbreak phrases (e.g. "DAN", "developer mode"), off-topic queries (recipes, sports, entertainment), and inputs that are too short or too long to be meaningful policy questions. Blocked requests are rejected with a clear reason message and the pipeline halts immediately.

Planner Agent

Receives the validated query and calls GPT-4o-mini with a tightly scoped system prompt to decompose the question into two to four concrete retrieval steps. The plan is rendered in the UI as part of the agent trace, making the system's reasoning transparent. The planner does not answer the question — it only structures the retrieval approach.

Retriever Agent

Performs keyword-overlap scoring over chunked local policy text files (refund policy, PII policy, escalation policy). Uses domain keyword hints to pre-select relevant files before scoring, improving both speed and precision. The top-K scoring chunks are passed downstream. No vector database or external search index is required.

Analyst Agent

Generates a draft answer using GPT-4o-mini, constrained by a system prompt that explicitly forbids using knowledge outside the retrieved policy context. If the context does not cover the question, the agent is instructed to say so rather than speculate. This is the core RAG step: retrieval-augmented generation grounded entirely in internal documents.

Critic Agent

Reviews the draft answer against the retrieved context using a structured evaluation schema. It outputs grounding status (Supported / Partially Supported / Not Supported), accuracy, completeness, a confidence level (HIGH / MEDIUM / LOW), and a recommendation of APPROVE, APPROVE WITH CAVEAT, or REVISE. This structured output drives the final formatting logic and any warning banners shown to the user.

Response Agent

Formats the approved answer into clear, plain-language prose suitable for a non-technical employee. It appends visible warning banners for low-confidence answers or answers flagged with a caveat, and always names the source policy documents at the end of the response.

4. Key Features

- **Input Safety:** Prompt injection blocking via regex patterns covering common injection and jailbreak phrases, executed before any LLM call.
- **Retrieval-Based Grounding:** Policy-grounded retrieval using keyword-overlap scoring over chunked local text files, with file-hint routing to reduce unnecessary search scope.
- **Answer Validation:** Critic validation before every user-facing response, with structured evaluation across grounding, accuracy, completeness, and confidence dimensions.
- **Confidence Indicators:** Confidence indicators displayed as coloured banners in the UI, derived from the Critic Agent output.
- **Explainability:** Full agent trace shown in the UI at every step — plan, retrieved chunks, analyst draft, critic evaluation, and final answer — providing end-to-end explainability.
- **Secure Key Handling:** API keys managed via python-dotenv, never logged or surfaced in any agent output or UI element.

5. Technologies Used

- Python 3.9 — core application logic and all agent modules
- Streamlit — web UI framework for the interactive front end and agent trace display
- OpenAI API (GPT-4o-mini) — language model for Planner, Analyst, Critic, and Response agents
- Local .txt policy files — document store for retrieval (no external database required)
- python-dotenv — environment variable management for API key loading
- Standard library only (re, pathlib, dataclasses) — no heavy orchestration framework dependencies

6. Challenges & Future Improvements

Challenges Encountered

Maintaining retrieval quality with keyword-based scoring was the primary challenge. Queries phrased differently from the policy document wording can produce lower-relevance chunks, which directly affects answer quality. Preventing hallucination required careful system prompt design — the Analyst agent needed explicit negative constraints, not just positive guidance, to reliably stay within retrieved context. Balancing the Critic Agent's strictness was also non-trivial: an overly strict critic caused unnecessary caveats on clear, well-grounded answers.

Future Improvements

- Replace keyword scoring with embedding-based semantic retrieval (e.g. sentence-transformers with FAISS) to improve recall on paraphrased queries.
- Implement conditional workflow branching so the Critic Agent's REVISE recommendation triggers a re-retrieval loop rather than a hard stop.
- Add an audit logging layer to record all queries, agent outputs, and critic decisions for compliance review.
- Expand the policy document library to cover HR, IT security, travel expense, and acceptable use policies.
- Add user role-based access controls to restrict visibility of certain policy sections.
- Introduce support for PDF-formatted policy documents via text extraction.

7. Conclusion

WorkBridge AI demonstrates that a useful, safe, and explainable multi-agent system can be built with a small, focused toolset. The architecture prioritizes reliability over complexity: each agent does one thing, the pipeline is fully sequential, guardrails run before any LLM call, and every step is visible to the user. The project shows practical enterprise system thinking — grounded answers, visible confidence levels, and blocked unsafe inputs — without overstating what a prototype can deliver. With semantic retrieval and conditional orchestration added, this architecture would be a solid foundation for a production enterprise policy assistant.